

# Lesson 10: Async in Dart

## Why waiting is a problem

Some work takes time. Reading a file, asking a server for data, waiting for the user to pick a date -- none of this happens at once.

If your app just stops and waits, the screen freezes. Buttons do not respond. The app feels dead. Users hate this.

Dart solves it with async code: work that runs in the background while the rest of your app keeps going. The screen stays alive, and your code is told when the work is done.

## Futures: a value that is on its way

When you start slow work, Dart does not give you the answer right away. It gives you a Future -- a promise that the value will come later.

```
Future<String> fetchUsername() async {  
  await Future.delayed(const Duration(seconds: 2));  
  return 'Aarav';  
}
```

Read it like this: `fetchUsername()` will take 2 seconds, and then give back a `String`. The `Future<String>` is like a receipt -- "your string is being prepared, come back later."

## async and await

Two keywords make async code easy to read:

- `async` -- marks a function as "this will do slow work."
- `await` -- means "wait here for the result, but let the app keep running."

```
void greetUser() async {  
  print('Fetching...');  
  final name = await fetchUsername(); // app stays alive while waiting  
  print('Hello, $name!');  
}
```

Without `await`, `name` would be a `Future`, not a `String`. With `await`, Dart pauses *this function* until the value arrives -- but the rest of your app (buttons, animations) keeps working.

A rule to remember: you can only use `await` inside a function marked `async`.

## Showing a loading spinner

While waiting, show the user that something is happening. The common pattern is a loading spinner:

```
class UserScreen extends StatefulWidget {
  const UserScreen({super.key});

  @override
  State<UserScreen> createState() => _UserScreenState();
}

class _UserScreenState extends State<UserScreen> {
  String? username; // null means "still loading"

  @override
  void initState() {
    super.initState();
    loadUser();
  }

  void loadUser() async {
    final name = await fetchUsername();
    setState(() {
      username = name; // now rebuild with the real data
    });
  }

  @override
  Widget build(BuildContext context) {
    if (username == null) {
      return const Center(child: CircularProgressIndicator());
    }
    return Center(child: Text('Hello, $username!'));
  }
}
```

The idea is simple: start with null, show a spinner. When the data arrives, call `setState()` and show the real UI. You will use this pattern in almost every app you build.

## When things go wrong

Slow work can fail. The server is down, the file is missing, the internet is off. Dart uses `try/catch` for this:

```
void loadUser() async {
  try {
    final name = await fetchUsername();
    setState(() {
      username = name;
    });
  } catch (e) {
    // Handle error
  }
}
```

```
    });  
  } catch (e) {  
    setState(() {  
      username = 'Guest'; // fallback when loading fails  
    });  
  }  
}
```

Put risky code inside try. If anything inside fails, Dart jumps to catch instead of crashing your app. Always wrap network calls and file reading in try/catch.

## Key terms

- Async -- work that runs in the background without freezing the app.
- Future -- a promise that a value will arrive later.
- async -- marks a function as doing slow work.
- await -- waits for a Future's value while the app keeps running.
- try/catch -- handles errors so the app does not crash.